# Programming with C++

## Abstract Machine

LSEG Technology

12th February 2026

Upeka Weerasekara

# What you'll need today

- Option 1 (Recommended – Windows):

  Visual Studio Community Edition https://visualstudio.microsoft.com/vs/community/

- Option 2 (Any OS):

  Visual Studio Code with any C++ compiler

- Option 3 (No setup – for practice):

  Online compiler like Programiz Online Compiler

(Scan QR code for Programiz)

# Agenda

- Abstract Machine
- Structs and Classes
- Virtual Functions
- Composition
- Move semantics
- Smart Pointers
- STL Containers
- Exception handling

# Session Overview

Already know about: types, functions, pointers: like alphabet.

Will try to combine letters to form words based on **underlying rules**

Two Perspectives

- The Machine
- The Programmer

# The C++ Abstract Machine (“The Rules”)

The C++ Abstract Machine is a conceptual model that defines how C++ programs execute.

It's not a real machine, but a specification that all C++ compilers must follow to ensure consistent program behavior.

KEY CONCEPTS:
1. Sequence of execution
2. Memory model
3. Observable behavior
4. The "as-if" rule
5. Undefined behavior

# Problem with loose variables

**Scenario**: Building a 2D game and maintain positions of players (x, y) and there are 100 players

**Approach you may not follow:** 2 x 100 number of variables (x1, x2, ….. , x100, y1, y2, …. y100)

**Problem**

We don’t have a rule for positions of a player to be together. Means nothing to abstract machine.

x1 and y1 could very well be far placed in memory space leading to Latency Issues

# Struct: Creating a “Package”

Group different variables under one single name

Defining a struct means creating a new data type for abstract machine to follow

Basically, wrapping data into a single unit

# Struct: Creating a “Package”

struct.cpp

```cpp
#include <iostream>

// Define a struct for a 2D point
struct Point {
    int x;
    int y;
};

int main() {
    // Create and initialize point objects
    struct Point p1;
    p1.x = 10;
    p1.y = 20;

    struct Point p2 = {30, 40};  // Initialize at declaration

    struct Point p3;
    p3.x = -5;
    p3.y = 15;

    // Print the points
    std::cout << "Point 1: (" << p1.x << ", " << p1.y << ")" << std::endl;
    std::cout << "Point 2: (" << p2.x << ", " << p2.y << ")" << std::endl;
    std::cout << "Point 3: (" << p3.x << ", " << p3.y << ")" << std::endl;

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - samples

```
[ec2-user@ip-10-230-18-64 samples]$ g++ -o struct struct.cpp
[ec2-user@ip-10-230-18-64 samples]$ ./struct
Point 1: (10, 20)
Point 2: (30, 40)
Point 3: (-5, 15)
```

# Memory Layout

**Contiguous Memory**: Abstract machine guarantees that members are placed side by side in a single block of memory

**Goal of the machine**: CPU wants to read memory as fast as possible. To do this, it prefers to read data in chunks (usually in 4 or 8 bytes at a time)

**Alignment and Padding**: When data doesn’t directly fit into chunks

# Memory Layout

```cpp
struct DataLayout {
    // char occupies 1 byte
    // Offset: 0
    char a;

    // Compiler inserts 3 bytes of padding here (bytes 1-3)
    // to align the next int on a 4-byte boundary
    // Offset: 4
    int b;

    // char occupies 1 byte
    // Offset: 8
    char c;

    // Compiler inserts 7 bytes of padding here (bytes 9-15)
    // Why 7 bytes? After char c at offset 8, the next member is double d.
    // double requires 8-byte alignment, so d must start at offset 16.
    // This requires 7 bytes of padding (9-15) to reach the next 8-byte boundary.
    // Alignment rule: each member must start at an offset divisible by its size.
    // to align the next double on an 8-byte boundary
    // Offset: 16
    double d;

    // char occupies 1 byte
    // Offset: 24
    char e;

    // Compiler inserts 7 bytes of padding at the end (bytes 25-31)
    // to make the struct size a multiple of its largest member alignment (8)
    // Offset: 32 (end of struct)
};
```

```cpp
int main() {
    std::cout << "Structure Memory Layout Analysis:" << std::endl;
    std::cout << "================================" << std::endl;

    // Display the total size of the struct
    std::cout << "Total size of DataLayout: " << sizeof(DataLayout) << " bytes" << std::endl;

    // Display offsets of each member
    DataLayout obj;
    std::cout << "\nMember offsets:" << std::endl;
    std::cout << "  a (char):   " << offsetof(DataLayout, a) << " bytes" << std::endl;
    std::cout << "  b (int):    " << offsetof(DataLayout, b) << " bytes" << std::endl;
    std::cout << "  c (char):   " << offsetof(DataLayout, c) << " bytes" << std::endl;
    std::cout << "  d (double): " << offsetof(DataLayout, d) << " bytes" << std::endl;
    std::cout << "  e (char):   " << offsetof(DataLayout, e) << " bytes" << std::endl;

    // Display individual member sizes
    std::cout << "\nMember sizes:" << std::endl;
    std::cout << "  sizeof(char):   " << sizeof(char) << " byte" << std::endl;
    std::cout << "  sizeof(int):    " << sizeof(int) << " bytes" << std::endl;
    std::cout << "  sizeof(double): " << sizeof(double) << " bytes" << std::endl;

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - sa

```
[ec2-user@ip-10-230-18-64 samples]$ ./memory
Structure Memory Layout Analysis:
================================
Total size of DataLayout: 32 bytes

Member offsets:
  a (char):   0 bytes
  b (int):    4 bytes
  c (char):   8 bytes
  d (double): 16 bytes
  e (char):   24 bytes

Member sizes:
  sizeof(char):   1 byte
  sizeof(int):    4 bytes
  sizeof(double): 8 bytes
```

# Transition to Classes: Adding Logic to Data

**Class** is simply a struct that can also hold functions (“methods”) which also knows **how to protect** member variables or methods

Not a mere “**Box of variables**” but an “**Entity**” that knows how to manage its own state

# Transition to Classes: Adding Logic to Data

```cpp
class Point {
private:
    // Private data members - can only be accessed within the class
    int x;
    int y;

    // Private member function - helper function only used internally
    // Calculates the squared distance from origin (avoids sqrt for efficiency)
    double distanceSquaredFromOrigin() {
        return x * x + y * y;
    }

public:
    // Public constructor - allows initialization of private members
    Point(int xVal, int yVal) {
        x = xVal;
        y = yVal;
    }

    // Public member function - can be called from outside the class
    // Displays the point coordinates and distance from origin
    void display() {
        std::cout << "Point: (" << x << ", " << y << ")";

        // Private member function can be called from within public functions
        double distSq = distanceSquaredFromOrigin();
        double distance = sqrt(distSq);

        std::cout << " - Distance from origin: " << distance << std::endl;
    }

    // Public getter methods - provide controlled access to private data
    int getX() { return x; }
    int getY() { return y; }

    // Public setter methods - allow controlled modification of private data
    void setX(int xVal) { x = xVal; }
    void setY(int yVal) { y = yVal; }
};
```

```cpp
int main() {
    // Create and initialize point objects using constructor
    Point p1(10, 20);
    Point p2(30, 40);
    Point p3(-5, 15);

    // Call public member function to display points
    // Note: We cannot directly access p1.x or p1.y because they are private
    p1.display();
    p2.display();
    p3.display();

    // Demonstrate using getter methods to access private data
    std::cout << "\nAccessing p1 coordinates using getters: ("
              << p1.getX() << ", " << p1.getY() << ")" << std::endl;

    // Demonstrate using setter methods to modify private data
    p1.setX(100);
    p1.setY(200);
    std::cout << "After modification: ";
    p1.display();

    /* KEY DIFFERENCES FROM STRUCT:
     * 1. Class members are private by default (struct: public by default)
     * 2. data is protected and accessed through methods
     * 3. Can have private helper functions like distanceSquaredFromOrigin()
     * 4. Constructor provides clean initialization syntax
     * 5. Getters/setters provide controlled access to private data
     */

    return 0;
}
```

# Life of an object

**Constructors:** Special functions called automatically when an object is born

**Destructors:** Special functions called automatically when an object is destroyed (goes out of scope)

```
class Book {
private:
    std::string title;
    int pages;

public:
    // Constructor - initializes the object
    Book(std::string t, int p) {
        title = t;
        pages = p;
        std::cout << "[Constructor] Created: '" << title << "'" << std::endl;
    }

    // Destructor - cleans up when object is destroyed
    ~Book() {
        std::cout << "[Destructor] Destroyed: '" << title << "'" << std::endl;
    }

    void display() {
        std::cout << "Book: '" << title << "' (" << pages << " pages)" << std::endl;
    }
};

int main() {
    std::cout << "Creating book1..." << std::endl;
    Book book1("C++ Basics", 200);
    book1.display();

    std::cout << "\nCreating book2..." << std::endl;
    Book book2("Advanced C++", 400);
    book2.display();

    std::cout << "\nEnd of main - objects will be destroyed now..." << std::endl;
    // Destructors called automatically in reverse order: book2, then book1

    return 0;
}
```

# Member Initialization: The faster way

**Concept:** Using the colon (:) list to set values before the constructor body runs

**Advantage:** Make easier to avoid uninitialized garbage data

```
// Example 2: Member Initializer List (Preferred Method)
class Rectangle_V2 {
private:
    int width;
    int height;

public:
    // Constructor using member initializer list (more efficient)
    // Members are directly initialized with the given values
    Rectangle_V2(int w, int h) : width(w), height(h) {
        // Constructor body can be empty or contain additional logic
        std::cout << "Rectangle created using initializer list" << std::endl;
    }

    void display() {
        std::cout << "Rectangle: " << width << " x " << height << std::endl;
    }
};
```

# Objects in Memory (Stack vs Heap)

**The Stack:** When a class variable is introduced normally, it lives on stack and disappears when out of scope

**The Heap:** If you use *new*, the object lives on Heap. It stays there until you manually *delete* it

# Virtual Functions

**Virtual functions allow a class to override behavior in a derived class and ensure the correct function is called based on the actual object type, not the pointer type.**

**When to Use Virtual Functions:**

- When you need different behavior in subclasses
- When you want to extend class families without changing existing code
- When you need true runtime polymorphism (not templates)

# Virtual Functions and vtable

**Why vtables exist**
When a class uses virtual functions, the compiler needs a way to pick the **correct overridden function at runtime**.
A vtable is the mechanism that enables this.

**What is a vtable?**
A **compiler-generated table of function pointers**.
Each entry corresponds to one virtual function in the class.
The table stores the addresses of the **most derived override** of each function.

**Where is the vtable stored?**
Usually stored once per class (not per object).
Its layout is compiler-specific, but the concept is universal.

**How objects access the vtable**
Each object of a class with virtual functions contains a hidden pointer (often called **vptr**).
The vptr points to the vtable of the actual class of the object.
This is how the runtime can determine which function to call.

**What happens during a virtual call?**
1.Object's vptr is read
2.vptr points to the class's vtable
3.The correct function entry is selected
4.That function is invoked

# Virtual Functions

```cpp
// Base class
class Animal {
public:
    // Virtual function - can be overridden by derived classes
    virtual void makeSound() {
        std::cout << "Animal makes a generic sound" << std::endl;
    }

    // Non-virtual function - cannot be overridden properly
    void sleep() {
        std::cout << "Animal is sleeping" << std::endl;
    }
};

// Derived class 1
class Dog : public Animal {
public:
    // Override the virtual function
    void makeSound() override {
        std::cout << "Dog says: Woof! Woof!" << std::endl;
    }

    void sleep() {
        std::cout << "Dog is sleeping on the couch" << std::endl;
    }
};
```

```cpp
int main() {
    std::cout << "=== Creating Objects ===" << std::endl;
    Dog dog;
    Cat cat;

    std::cout << "\n=== Direct Calls (no polymorphism) ===" << std::endl;
    dog.makeSound();
    cat.makeSound();

    std::cout << "\n=== Using Base Class Pointers (polymorphism) ===" << std::endl;
    // Base class pointer pointing to derived class objects
    Animal* ptr1 = &dog;
    Animal* ptr2 = &cat;

    // Virtual function: calls the DERIVED class version (runtime polymorphism)
    std::cout << "ptr1->makeSound(): ";
    ptr1->makeSound();  // Calls Dog::makeSound()
```

# Inheritance Trap

**Deep "Family Trees":** A -> B -> C -> D -> E

**Places to Avoid**

- Fragility: Changing grandparent class (A) causes changes to all child classes
- Complexity to track where functions inherit from
- More Vtable lookups and padding

# Composition: Has-a vs Is-a

**Composition is about Nesting in contrast to Branching in inheritance**

Inside memory, composition looks like boxes inside a box

Since parts are closer in memory, the CPU can process them much faster.

Composition in preferred in cases relate to lose coupling

# Composition: Has-a vs Is-a

```cpp
// ===== INHERITANCE Approach =====
class Bird {
public:
    virtual void move() {
        std::cout << "Bird is flying" << std::endl;
    }
};

class Penguin : public Bird {
public:
    void move() override {
        std::cout << "Penguin is waddling (can't fly!)" << std::endl;
    }
};
// Problem: Not all birds can fly, but inheritance forces this relationship
```

```cpp
// Animal using composition - more flexible
class Animal {
private:
    std::string name;
    FlyBehavior* flyBehavior;   // HAS-A flying ability (optional)
    WalkBehavior* walkBehavior; // HAS-A walking ability (optional)

public:
    Animal(std::string n, FlyBehavior* fb, WalkBehavior* wb)
        : name(n), flyBehavior(fb), walkBehavior(wb) {}

    void move() {
        std::cout << name << ": ";
        if (flyBehavior) {
            flyBehavior->fly();
        } else if (walkBehavior) {
            walkBehavior->walk();
        }
    }
};
```

# R – Value References

**Lvalue (left value):** An expression that refers to a memory location and allows us to take its address. Has a name or persistent identity.

Can appear on the left side of an assignment.

Examples: variables, array elements, dereferenced pointers, function return by reference

**Rvalue (right value):** An expression that represents a temporary value that doesn't persist beyond the expression.

Cannot take its address (no persistent location)Can only appear on the right side of an assignment.

Examples: literals (5, 3.14), temporary objects, function return by value, expression results (a + b)

```
int x = 10;       // x is lvalue, 10 is rvalue
int* p = &x;      // OK: x is lvalue (has address)
// int* p = &10; // ERROR: 10 is rvalue (no address)

x = 20;           // x is lvalue (can be assigned to)
// 10 = x;        // ERROR: 10 is rvalue (can't assign to it)

int y = x + 5;    // (x + 5) is rvalue (temporary result)
```

# Move Semantics

**Problem:** When you return a large object from a function, Old C++ way was to copy every single item to a new location and then delete the old ones leading to a massive waste of CPU time.

**Solution:** Instead of copying data, we move the pointers from old object to new object

```
// Move Constructor - TRANSFERS ownership (cheap)
SimpleString(SimpleString&& other) noexcept {
    // "Steal" the data from other
    data = other.data;
    size = other.size;

    // Leave other in a valid but empty state
    other.data = nullptr;
    other.size = 0;

    std::cout << "[Move Constructor] Moved: \"" << data << "\"" << std::endl;
}
```

```
std::cout << "\n=== Example 2: Move Constructor ===" << std::endl;
SimpleString s3("Temporary");
SimpleString s4 = std::move(s3);  // Move constructor called
std::cout << "After move:" << std::endl;
s3.print();  // s3 is now empty
s4.print();  // s4 owns the data
```

# Smart Pointers: unique pointer

**Speed of a pointer but safety of a local variable**

**A wrapper around a raw pointer**

**Automatically cleans up when goes out of scope. It automatically calls delete**

# Smart Pointers: unique pointer

```cpp
// ===== UNIQUE_PTR BENEFITS =====
void uniquePtrExample() {
    std::cout << "\n=== unique_ptr (Automatic Management) ===" << std::endl;

    std::unique_ptr<Resource> ptr = std::make_unique<Resource>(3);
    ptr->use();

    // No delete needed! Automatic cleanup when ptr goes out of scope
}  // Resource automatically destroyed here
```

unique_ptr.cpp

```cpp
int main() {

    uniquePtrExample();
    ownershipExample();
    passingOwnershipExample();

    std::cout << "\n=== End of main ===" << std::endl;

    return 0;
}

/* KEY DIFFERENCES:
 *
 * RAW POINTERS:
 * - Manual memory management (new/delete)
 * - Easy to forget delete → memory leaks
 * - Can use after delete → undefined behavior
 * - Can delete twice → crash
 * - No ownership semantics
 *
 * UNIQUE_PTR:
 * - Automatic memory management (RAII)
 * - No memory leaks (automatic cleanup)
 * - Cannot use after move (becomes nullptr)
 * - Cannot be copied (exclusive ownership)
 * - Clear ownership semantics
 *
 * WHEN TO USE unique_ptr:
 * - Single ownership of dynamically allocated objects
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[ec2-user@ip-10-230-18-64 samples]$ ./up
[Resource 2] Being used
--- Leak happened (Resource 2 not destroyed) ---

=== unique_ptr (Automatic Management) ===
[Resource 3] Created
[Resource 3] Being used
[Resource 3] Destroyed
```

# Smart Pointers: shared pointer

Shared pointer allows multiple pointers to point to the same object

Abstract machine maintains a hidden counter on pointers referenced to shared pointer

Object in memory is deleted when the very last shared pointer is destroyed and count reaches 0

# Smart Pointers: shared pointer

shared_ptr.cpp

```cpp
int main() {
    std::cout << "=== Shared Ownership ===" << std::endl;

    // Create shared_ptr - reference count = 1
    std::shared_ptr<Resource> ptr1 = std::make_shared<Resource>(100);
    std::cout << "ptr1 use_count: " << ptr1.use_count() << std::endl;

    {
        // Multiple pointers can share ownership
        std::shared_ptr<Resource> ptr2 = ptr1;  // reference count = 2
        std::cout << "After ptr2 = ptr1:" << std::endl;
        std::cout << "  ptr1 use_count: " << ptr1.use_count() << std::endl;
        std::cout << "  ptr2 use_count: " << ptr2.use_count() << std::endl;

        std::shared_ptr<Resource> ptr3 = ptr1;  // reference count = 3
        std::cout << "After ptr3 = ptr1:" << std::endl;
        std::cout << "  ptr1 use_count: " << ptr1.use_count() << std::endl;

        ptr2->display();

    } // ptr2 and ptr3 go out of scope, but Resource NOT destroyed yet
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - samples

```
[ec2-user@ip-10-230-18-64 samples]$ ./up

=== End of main ===
[ec2-user@ip-10-230-18-64 samples]$ g++ -o shared shared_ptr.cpp
[ec2-user@ip-10-230-18-64 samples]$ ./shared
=== Shared Ownership ===
Resource(100) constructed
ptr1 use_count: 1
After ptr2 = ptr1:
  ptr1 use_count: 2
  ptr2 use_count: 2
After ptr3 = ptr1:
  ptr1 use_count: 3
Resource value: 100
```

shared_ptr.cpp

```cpp
int main() {
    } // ptr2 and ptr3 go out of scope, but Resource NOT destroyed yet

    std::cout << "\nAfter inner block:" << std::endl;
    std::cout << "  ptr1 use_count: " << ptr1.use_count() << std::endl;
    std::cout << "  Resource still alive!\n";
    ptr1->display();

    // Reset manually
    std::cout << "\n=== Manual reset ===" << std::endl;
    ptr1.reset();  // Decrements count to 0, Resource destroyed
    std::cout << "ptr1 after reset: " << (ptr1 ? "valid" : "null") << std::endl;

    // Another example
    std::cout << "\n=== New shared_ptr ===" << std::endl;
    auto ptr4 = std::make_shared<Resource>(200);
    std::cout << "Created ptr4\n";

    return 0;
    // ptr4 goes out of scope, Resource(200) destroyed automatically
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - samples

```
[ec2-user@ip-10-230-18-64 samples]$ ./shared
  ptr2 use_count: 2
After ptr3 = ptr1:
  ptr1 use_count: 3
Resource value: 100

After inner block:
  ptr1 use_count: 1
  Resource still alive!
Resource value: 100

=== Manual reset ===
Resource(100) destroyed
ptr1 after reset: null

=== New shared_ptr ===
Resource(200) constructed
Created ptr4
Resource(200) destroyed
```

# Containers

**Container is an object that stores other objects. The abstract machine ensures that these containers manage their own memory**

**Frequently used Containers:**

1. **std::vector**
2. **std::array**
3. **std::set**
4. **std::map**

# try-catch: exception handling

**Concept:** Exceptions are for exceptional events such as out of memory, file not found, out of bounds etc

When an error is thrown, abstract machine **stops current function and looks "up the stack" for a catch block**

**If an exception is thrown, destructors of all objects are still called**

```
try {
    int result = divide(10, 2);
    std::cout << "10 / 2 = " << result << std::endl;

    result = divide(10, 0);  // This will throw
    std::cout << "This won't print" << std::endl;
}
catch (const std::runtime_error& e) {
    std::cout << "Caught exception: " << e.what() << std::endl;
}
```

# Mini Exercise

**Create a std::vector of strings**

**Add 3 Names**

**Use a try-catch block to try and access 10th element using .at(10)**

**Print all 3 names**

# References

- Abstract Machine
- Move Semantics
- Smart Pointers
- lvalue and rvalue
- Containers

# Thank You

upeka.weerasekara@lseg.com